Soluciones del boletín de ejercicios

En los ejercicios de diseño se ofrece una solución modelo posible; las alternativas coherentes también son válidas.

I. Modelado DL y razonamiento integrado

Ejercicio 1. Plataforma de formación profesional

a) TBox y características:

• Estudiante ⊑ Persona; Profesor ⊑ Persona; CursoAvanzado ⊑ Curso.

• EstudianteAvanzado ≡ Estudiante ⊓ ∃matriculadoEn.CursoAvanzado.

• EstudianteCertificado ≡ Estudiante ⊓ (≥ 2 obtieneCertificado.Certificado).

• Tutor ≡ Profesor ⊓ ∃tutoriza.Estudiante.

• ∃matriculadoEn.⊤ ⊑ Estudiante; ⊤ ⊑ ∀matriculadoEn.Curso.

• ∃imparte.⊤ ⊑ Profesor; ⊤ ⊑ ∀imparte.Curso.

• ∃tutoriza.⊤ ⊑ Profesor; ⊤ ⊑ ∀tutoriza.Estudiante.

• Funcional(imparte); tieneMatriculado ≡ matriculadoEn⁻.

b) De matriculadoEn(laura,cursoIA), Laura es Estudiante y Persona; al ser cursoIA CursoAvanzado, Laura es EstudianteAvanzado. Miguel es Profesor y Persona por imparte y tutoriza; también Tutor, pues tutoriza a Laura, que es Estudiante. IMPORTANTE: no se deduce EstudianteCertificado(laura) con la TBox estricta: no se ha declarado que c1 y c2 sean Certificado ni se ha especificado rango de obtieneCertificado. Si se añade rango Certificado (o ambos tipos), sí se infiere.

c) cursoIA es CursoAvanzado y Curso; cursoDatos es Curso por el rango de imparte. La ontología completa es inconsistente, por lo que estas son las inferencias explicativas previas al conflicto, no un conjunto de respuestas fiable de un razonador bajo semántica clásica.

d) tieneMatriculado(cursoIA,laura).

e) Inconsistente: Funcional(imparte), imparte(miguel,cursoIA), imparte(miguel,cursoDatos) fuerzan cursoIA = cursoDatos, en contradicción con DifferentIndividuals(cursoIA,cursoDatos).

f) Sin la diferencia explícita, el razonador puede identificar cursoIA y cursoDatos; la ontología vuelve a ser consistente, siempre que no existan otras restricciones contradictorias.

Ejercicio 2. Red de colaboración científica

a) Investigador ⊑ Persona; Simétrica(colaboraCon); Transitiva(supervisaA); Asimétrica(supervisaA); ∃perteneceA.⊤ ⊑ Investigador; ⊤ ⊑ ∀perteneceA.Centro; Investigador ⊑ ∃trabajaCon.Self (reflexividad local).

b) Simetría: colaboraCon(bruno,ana), colaboraCon(carla,bruno). Transitividad: supervisaA(ana,carla), supervisaA(bruno,diego), supervisaA(ana,diego).

c) No se deduce colaboraCon(ana,carla), porque colaboraCon no es transitiva. Tampoco supervisaA(diego,ana); sería incompatible con la asimetría y la cadena ya afirmada.

d) trabajaCon(ana,ana), trabajaCon(bruno,bruno), trabajaCon(carla,carla) y trabajaCon(diego,diego).

e) La relación añadida cierra un ciclo; por transitividad se derivan autorrelaciones y relaciones opuestas, incompatibles con asimetría. Inconsistencia en la interpretación lógica de los axiomas.

f) ana es Investigador y Persona; centroNorte es Centro por el rango de perteneceA.

Ejercicio 3. Sistema de acceso a instalaciones

a) Empleado(lucia) y Empleado ⊑ Persona ⇒ Persona(lucia). ZonaSegura(laboratorio) y ZonaSegura ⊑ Zona ⇒ Zona(laboratorio). La relación de acceso y la definición equivalente implican EmpleadoAutorizado(lucia).

b) Por simetría: conectaCon(zonaB,zonaA) y conectaCon(zonaC,zonaB). Por transitividad: esSuperiorA(jefe,tecnico). No se deduce conectaCon(zonaA,zonaC).

c) conectaCon(zonaA,zonaA) contradice la irreflexividad: ontología inconsistente.

d) esSuperiorA(tecnico,jefe) contradice la asimetría, porque ya se infiere esSuperiorA(jefe,tecnico). Además crea un ciclo con la transitividad.

e) No. Irreflexiva prohíbe R(x,x), pero permite R(x,y) y R(y,x) para x≠y. Asimétrica prohíbe ambas direcciones a la vez y, por tanto, implica irreflexividad.

Ejercicio 4. Biblioteca digital

a) LibroTecnico ⊑ Libro; AutorProductivo ≡ Autor ⊓ (≥3 escribe.Libro); UsuarioTecnico ≡ Usuario ⊓ ∃tomaPrestado.LibroTecnico; ∃escribe.⊤ ⊑ Autor; ⊤ ⊑ ∀escribe.Libro; ∃tomaPrestado.⊤ ⊑ Usuario; ⊤ ⊑ ∀tomaPrestado.Libro; ∃correspondeA.⊤ ⊑ Prestamo; ⊤ ⊑ ∀correspondeA.Usuario.

b) sofia: Autor; diego: Usuario; l1,l2,l3: Libro; l2 también LibroTecnico; prestamo7: Prestamo.

c) AutorProductivo(sofia) por tres libros distintos; UsuarioTecnico(diego) por préstamo de l2, que es LibroTecnico.

d) No. Sin DifferentIndividuals, tres nombres pueden denotar menos de tres objetos distintos; no queda garantizada la cardinalidad.

e) No. Bajo OWA, que el único libro conocido sea técnico no permite afirmar que todos los libros prestados sean técnicos. Se necesita una restricción universal explícita o un cierre local suficiente.

Ejercicio 5. Red de alojamientos turísticos

a) TBox:

• Hotel ⊑ Alojamiento; Apartamento ⊑ Alojamiento; Hotel ⊓ Apartamento ⊑ ⊥; CiudadExtranjera ⊑ Ciudad.

• ClienteFrecuente ≡ Cliente ⊓ (≥3 realizaReserva.Reserva).

• ReservaInternacional ≡ Reserva ⊓ ∃reservaAlojamiento.(Alojamiento ⊓ ∃situadoEn.CiudadExtranjera).

• ReservaHotel ≡ Reserva ⊓ ∃reservaAlojamiento.Hotel.

• ∃realizaReserva.⊤ ⊑ Cliente; ⊤ ⊑ ∀realizaReserva.Reserva.

• ∃reservaAlojamiento.⊤ ⊑ Reserva; ⊤ ⊑ ∀reservaAlojamiento.Alojamiento.

• ∃situadoEn.⊤ ⊑ Alojamiento; ⊤ ⊑ ∀situadoEn.Ciudad.

• Simétrica(compatibleCon); Alojamiento ⊑ ∃compatibleCon.Self (reflexividad solo para alojamientos).

• Simétrica(incompatibleCon); Irreflexiva(incompatibleCon).

b) maria: Cliente, ClienteFrecuente. r1: Reserva, ReservaInternacional y ReservaHotel. hotelSol: Hotel y Alojamiento; además pertenece a ∃situadoEn.CiudadExtranjera. paris: CiudadExtranjera y Ciudad.

c) Sí, porque r1, r2 y r3 son reservas por el rango de realizaReserva y son tres individuos distintos. Sin diferencia explícita no se garantiza el mínimo de tres.

d) Sí. r1 reserva hotelSol, que es Hotel, y hotelSol está situado en paris, que es CiudadExtranjera.

e) compatibleCon(aptoMar,hotelSol), compatibleCon(hotelSol,hotelSol), compatibleCon(aptoMar,aptoMar). incompatibleCon(aptoMar,hotelSol).

f) No necesariamente. Las propiedades compatibleCon e incompatibleCon no han sido declaradas mutuamente excluyentes.

g) Inconsistencia inmediata por Irreflexiva(incompatibleCon).

h) Inconsistencia porque Hotel(hotelSol) y Apartamento(hotelSol) contradicen la disjointness entre ambas clases.

II. Reglas SWRL

Ejercicio 6. Recomendaciones audiovisuales

a) haVisto(?u,?p) ∧ dirigidaPor(?p,?d) → sigueDirector(?u,?d).

b) sigueDirector(?u,?d) ∧ dirigidaPor(?p2,?d) → recomienda(?u,?p2).

c) No puede añadirse simplemente “no haVisto(?u,?p2)” como negación por fallo: SWRL estándar es monotónica y OWA no convierte ausencia en falsedad. Solución de diseño: registrar explícitamente una propiedad elegibleParaRecomendacion(u,p) calculada por una aplicación con información cerrada, y usar sigueDirector(?u,?d) ∧ dirigidaPor(?p,?d) ∧ elegibleParaRecomendacion(?u,?p) → recomienda(?u,?p). Otra alternativa es filtrar los resultados después del razonamiento usando la base de datos de visualizaciones completa.

d) Ambas reglas enlazan individuos a través de relaciones distintas y generan nuevas relaciones entre ellos; una simple restricción existencial DL clasificaría individuos, pero no expresa directamente esas conexiones en el vocabulario de clases habitual.

Ejercicio 7. Alertas de sensores

a) Sensor(?s) ∧ temperatura(?s,?t) ∧ greaterThan(?t,40) → requiereRevision(?s,true).

b) instaladoEn(?s,?h) ∧ temperatura(?s,?t) ∧ greaterThan(?t,40) → alertaHabitacion(?h,?s).

c) Sensor(?s) ∧ temperatura(?s,?t) ∧ humedad(?s,?u) ∧ greaterThan(?t,35) ∧ greaterThan(?u,80) → requiereRevision(?s,true).

Ejercicio 8. Relaciones profesionales

a) participaEn(?x,?p) ∧ participaEn(?y,?p) ∧ differentFrom(?x,?y) → conoceProfesionalmente(?x,?y).

b) Se requiere una diferencia demostrable entre x e y, por ejemplo DifferentIndividuals(ana,bruno); dos nombres distintos no bastan por sí solos.

c) trabajaEn(?x,?e1) ∧ trabajaEn(?y,?e2) ∧ participaEn(?x,?p) ∧ participaEn(?y,?p) ∧ differentFrom(?e1,?e2) → colaboracionInterempresa(?x,?y). Puede añadirse differentFrom(?x,?y) si se quiere exigir personas distintas.

d) Por OWA, no conocer trabajaEn(x,e) no prueba que sea falsa; SWRL estándar no usa negación por fallo.

Ejercicio 9. Elección entre DL y SWRL

• a) DL: VehiculoElectrico ⊑ Vehiculo.

• b) DL: ClientePremium ≡ Cliente ⊓ (≥5 realizaCompra.Compra), suponiendo que Compra y la relación están definidas; se necesita distinción de compras.

• c) SWRL: trabajaEn(?x,?e) ∧ participaEn(?e,?p) → vinculadoA(?x,?p).

• d) DL/OWL: Vehiculo ⊑ (≤1 tieneMatricula.⊤); alternativamente, funcionalidad global si la propiedad solo aplica a vehículos.

• e) SWRL: instaladoEn(?s,?h) ∧ temperatura(?s,?t) ∧ greaterThan(?t,50) → alertaEn(?h,?s).

• f) SWRL: participaEnSesion(?x,?s) ∧ participaEnSesion(?y,?s) ∧ differentFrom(?x,?y) → haCoincididoCon(?x,?y).

• g) DL: Medico ⊓ Paciente ⊑ ⊥.

III. Diseño de ontologías — soluciones modelo

Ejercicio 10. Congresos científicos

a) Jerarquía: Persona con roles contextuales Autor, Revisor y Chair (no necesariamente disjuntos); Articulo con ArticuloAceptado y ArticuloRechazado; Revision, AreaTematica, Sesion, Presentacion. PresentacionOral ⊑ Presentacion opcional.

b) Propiedades: tieneAutor(Articulo,Persona), autorContacto(Articulo,Persona), areaDe(Articulo,AreaTematica), especializadoEn(Revisor,AreaTematica), asignadoA(Revisor,Articulo), revisionDe(Revision,Articulo), realizadaPor(Revision,Revisor), presentaArticulo(Presentacion,Articulo), contienePresentacion(Sesion,Presentacion), tieneChair(Sesion,Chair). Datos: titulo(Articulo,string), puntuacion(Revision,integer), fecha(Sesion,dateTime).

c) ArticuloAceptado ⊓ ArticuloRechazado ⊑ ⊥; Sesion ⊓ Articulo ⊑ ⊥. No hacer Autor, Revisor y Chair disjuntos: una misma persona puede desempeñar varios roles.

d) ArticuloConRevision ≡ Articulo ⊓ ∃revisionInversa.Revision, donde revisionInversa ≡ revisionDe⁻. ArticuloMultiautor ≡ Articulo ⊓ (≥2 tieneAutor.Persona). SesionConPresentaciones ≡ Sesion ⊓ ∃contienePresentacion.Presentacion. Si se desea ArticuloAceptado por datos de estado, se requiere axiomatizar explícitamente cómo se representa ese estado.

ArticuloRevisado = con 3 revisiones

Articulo_aceptado= con 3 notas por encima de 5

e) Articulo ⊑ (=1 autorContacto.Persona) si la política exige exactamente uno; Funcional(tieneChair) para como máximo un chair por sesión, o Sesion ⊑ (=1 tieneChair.Chair) si se exige exactamente uno.

f) ABox: Persona(ana), Persona(luis), Persona(marta), Articulo(a1), Articulo(a2), Revision(rev1), Sesion(s1), Presentacion(p1), Chair(marta), tieneAutor(a1,ana), tieneAutor(a1,luis), DifferentIndividuals(ana,luis), revisionDe(rev1,a1), contienePresentacion(s1,p1), presentaArticulo(p1,a1), tieneChair(s1,marta). Inferencias: ArticuloMultiautor(a1); ArticuloConRevision(a1); SesionConPresentaciones(s1).

g) Consultas: ¿qué artículos son multiautor? ¿Qué sesiones tienen presentaciones y quién las preside?

Ejercicio 11. Comercio electrónico

a) Jerarquía textual: Producto → ProductoFisico, ProductoDigital; Pedido → PedidoConFisicos, PedidoSoloDigital, PedidoGrande; Cliente → ClienteFrecuente; entidades independientes Categoria, LineaPedido, Pago, Envio. Pedido y Producto no son subclases uno de otro.

b) ProductoFisico y ProductoDigital son clases; las categorías concretas pueden ser individuos de Categoria (si se requiere clasificación flexible). precio, unidades, importe y fecha son valores de datos; los pedidos, productos y clientes son individuos.

c) realizaPedido: Cliente→Pedido (inversa pedidoDe); tieneLinea: Pedido→LineaPedido; productoDeLinea: LineaPedido→Producto; cantidad: LineaPedido→integer; tienePago: Pedido→Pago; tieneEnvio: Pedido→Envio; perteneceCategoria: Producto→Categoria. Pedido ⊑ (=1 tienePago.Pago) si cada pedido tiene exactamente un pago. La presencia de LineaPedido evita confundir cantidad de unidades con número de productos distintos.

d) PedidoConFisicos ≡ Pedido ⊓ ∃tieneLinea.(LineaPedido ⊓ ∃productoDeLinea.ProductoFisico). PedidoConDigitales ≡ Pedido ⊓ ∃tieneLinea.(LineaPedido ⊓ ∃productoDeLinea.ProductoDigital). PedidoGrande ≡ Pedido ⊓ (≥3 tieneLinea.LineaPedido). ClienteFrecuente ≡ Cliente ⊓ (≥3 realizaPedido.Pedido). PedidoSoloDigital ≡ Pedido ⊓ ∀tieneLinea.(LineaPedido ⊓ ∀productoDeLinea.ProductoDigital); esta última exige cierre local para clasificar instancias desde listas conocidas.

e) ABox: Cliente(ana), Cliente(bruno), Cliente(celia); ProductoFisico(p1), ProductoFisico(p2), ProductoDigital(p3), ProductoDigital(p4), ProductoFisico(p5); Pedido(o1), Pedido(o2), Pedido(o3), Pedido(o4); LineaPedido(l1)…LineaPedido(l6); realizaPedido(ana,o1), realizaPedido(ana,o2), realizaPedido(ana,o3), realizaPedido(bruno,o4); tieneLinea(o1,l1), tieneLinea(o1,l2), tieneLinea(o1,l3), tieneLinea(o2,l4), tieneLinea(o3,l5), tieneLinea(o4,l6); productoDeLinea(l1,p1), productoDeLinea(l2,p3), productoDeLinea(l3,p5), productoDeLinea(l4,p4), productoDeLinea(l5,p2), productoDeLinea(l6,p3). Declarar distintos o1,o2,o3 y l1,l2,l3.

f) Inferencias: ClienteFrecuente(ana), PedidoGrande(o1), PedidoConFisicos(o1), PedidoConDigitales(o1), PedidoConDigitales(o2); estas dependen de los hechos y distinciones indicados.

g) No se infiere PedidoSoloDigital(o2) solo porque su única línea conocida contiene p4 digital. Bajo OWA pueden existir otras líneas. Cerrar localmente tieneLinea(o2) con {l4} y productoDeLinea(l4) con {p4}, o imponer restricciones universales explícitas apropiadas.

h) Regla comercial: importeTotal(?o,?x) ∧ greaterThan(?x,100) ∧ pedidoDe(?o,?c) → candidatoDescuento(?c,?o). candidatoDescuento es una nueva relación; los importes deben estar representados explícitamente o calculados fuera de OWL.

Ejercicio 12. Turismo inteligente

a) Jerarquías: RecursoTuristico → Museo, Monumento, EspacioNatural; Alojamiento → Hotel, Apartamento; Actividad → ActividadCultural, ActividadNaturaleza; Persona → Viajero; entidades independientes Destino, Reserva, ServicioAccesibilidad. Las subclases de recursos pueden declararse disjuntas solo si el dominio lo justifica.

b) ubicadoEn: RecursoTuristico→Destino; ofreceActividad: Destino→Actividad; disponeDe: Alojamiento→ServicioAccesibilidad; requiereServicio: Viajero→ServicioAccesibilidad; realizaReserva: Viajero→Reserva (inversa reservaDe); reservaAlojamiento: Reserva→Alojamiento. Reserva ⊑ (=1 reservaAlojamiento.Alojamiento) si cada reserva es de un único alojamiento. Los dominios/rangos se formalizan como ∃R.⊤ ⊑ Dominio y ⊤ ⊑ ∀R.Rango.

c) MuseoAccesible ≡ Museo ⊓ ∃disponeDe.Rampa (si disponeDe también se aplica a recursos). AlojamientoAccesible ≡ Alojamiento ⊓ ∃disponeDe.ServicioAccesibilidad. ViajeroConReserva ≡ Viajero ⊓ ∃realizaReserva.Reserva. DestinoCultural ≡ Destino ⊓ ∃ofreceActividad.ActividadCultural. Para evitar ambigüedad, Rampa se trata aquí como clase de servicio, no como individuo.

d) ABox: Museo(m1), disponeDe(m1,r1), Rampa(r1), Alojamiento(h1), disponeDe(h1,r2), ServicioAccesibilidad(r2), Viajero(ana), realizaReserva(ana,res1), Reserva(res1), Destino(d1), ofreceActividad(d1,act1), ActividadCultural(act1). Inferencias: MuseoAccesible(m1), AlojamientoAccesible(h1), ViajeroConReserva(ana), DestinoCultural(d1).

e) Declarar Hotel ⊓ Apartamento ⊑ ⊥, Hotel(h1) y Apartamento(h1) provoca inconsistencia. El conflicto se debe a la disjointness y a ambas aserciones de clase.

f) Regla SWRL: requiereServicio(?v,?s) ∧ disponeDe(?a,?s) → alojamientoCandidato(?v,?a). La regla conecta individuos a través de dos relaciones.

g) Consultas: ¿qué alojamientos son accesibles? ¿Qué destinos ofrecen actividades culturales? ¿Qué viajeros tienen reservas? ¿Qué recursos ofrecen rampa? ¿Qué alojamientos son candidatos para un viajero concreto?